Number Of Ways To Make Change

AlgoExpert · Medium · Dynamic Programming

The problem

- Count combinations of unlimited coins that sum to n.

Intuition

- Count combinations by processing one coin at a time so order doesn't matter.

Brute force vs optimal

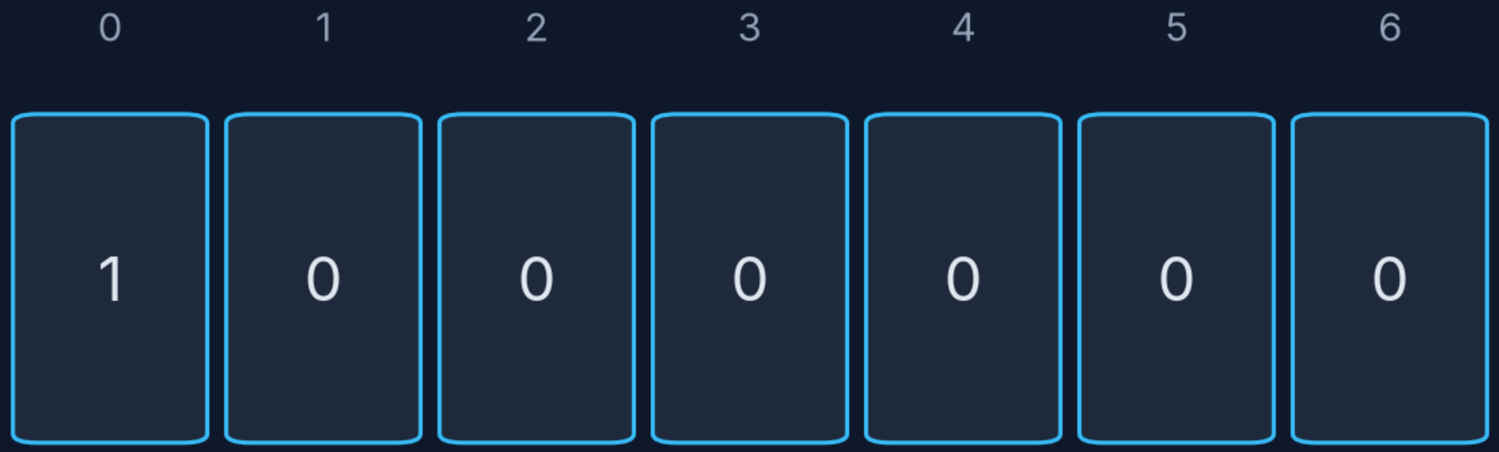
- Recursion over coins → exponential and double-counts orders
- DP with coins in the OUTER loop → counts combinations
- O(n · d) time, O(n) space

Approach

- 1. ways of zero is one
- 2. For each coin, for each amount, add ways of amount minus coin
- 3. Return ways of n

Worked example

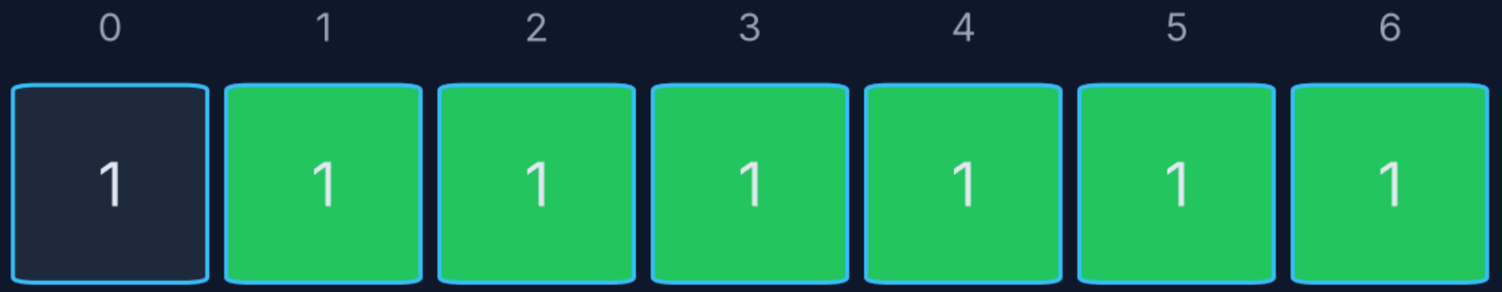
ways to make 0...6



ways[0] = 1 (use no coins)

Worked example

ways to make 0...6

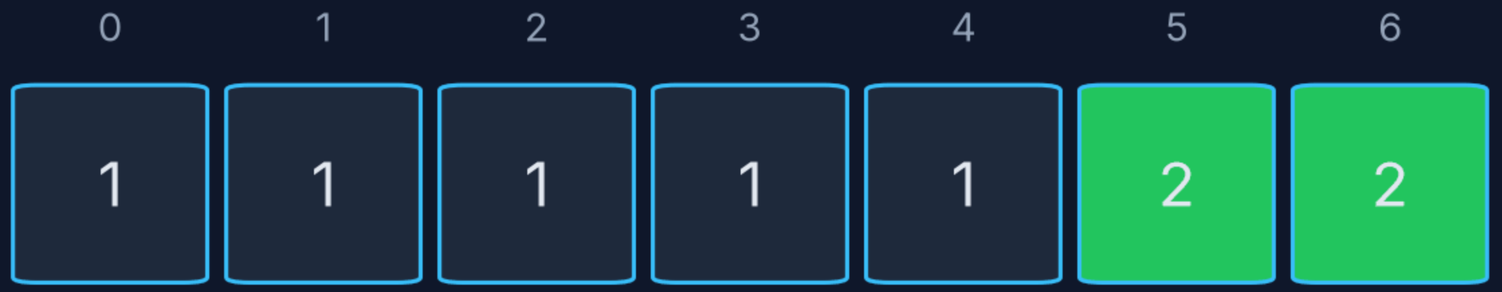



```
after coin 1: ways[a] += ways[a - 1]
```

After coin 1: [1, 1, 1, 1, 1, 1, 1]

Worked example

ways to make 0...6




```
after coin 5: ways[a] += ways[a - 5]
```

After coin 5: [1, 1, 1, 1, 1, 2, 2]

Java solution

```java
public class NumberOfWaysToMakeChange {
    public static int numberOfWaysToMakeChange(int n, int[] denoms) {
        int[] ways = new int[n + 1];
        ways[0] = 1;
        for (int d : denoms) for (int a = d; a <= n; a++) ways[a] += ways[a - d];
        return ways[n];
    }
}
```

Complexity

- Time: O(n*d)
- Space: O(n)

Edge cases & pitfalls

- Loop order matters: amounts outer → counts permutations
- Ways[0] = 1
- Large counts may overflow

Interview tips & follow-ups

- Explain combinations vs permutations
- Follow-up: limited coin supply

Recap

- Count combinations by processing one coin at a time so order doesn't matter.
- Complexity: O(n*d)